# GEOG5415M Week 5: Using Python as a GIS (answers)

October 21, 2026

**GEOG5415M Programming for Spatial Data Science**

## 1 Week 5: Using Python as a GIS (answers)

**Contact: N.S.Malleson@leeds.ac.uk**

## 2 Part A: Getting started with spatial data

Up until now all the packages we have needed have already been installed in the default Google Colab environment. This week we need some packages that are not: contextily (for basemaps), mapclassify (for classifying data in choropleth maps) and geo_northarrow (for adding a north arrow).

To install packages from within a notebook the quickest way is to use: !pip install PACKAGENAME

- the ! character is used to execute shell commands directly from a code cell. This allows you to run commands that you would normally run in a terminal or command prompt.
- pip is a Python package installer. It's used to install and manage software packages written in Python.
- install: a command for pip that tells it to install a package
- PACKAGENAME e.g. contextily or mapclassify: examples of packages we can install

If a package is already installed, pip will simply tell you so.

[ ]:
```
# Install the packages that are not in the default Colab environment.
# In Colab, run this cell at the start of every session (installed packages are␣
 ↪lost when the session ends).
# On your own computer you only need to run it once.
!pip install contextily mapclassify
!pip install git+https://github.com/pmdscully/geo_northarrow.git
```

As is the custom when coding, we import the required packages at the beginning Import the packages below

[ ]:
```
#Import the required packages

import pandas as pd
import numpy as np
import geopandas as gpd
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# for adding basemaps
import contextily as ctx

# for adding a north arrow to maps
from geo_northarrow import add_north_arrow

# set seaborn plotting theme to white
sns.set_theme(style="white")
```

We have imported a new package 'geopandas'. Geopandas works like pandas but also handles spatial data. Geopandas was designed to allow people to easily handle and use spatial datasets in Python.

## 2.1 Reading in spatial data

Geopandas data frames work the same way as pandas data frames, but are also able to handle the spatial element of the dataset. Therefore, the code we use is very similar. We use geopandas.read_file(file_path) replacing file_path with the path (or URL) of the spatial data file you want to read (in this case .geojson files).

We are going to read the files from the GitHub repo. The urls to the data on GitHub are: - https://github.com/MSc-Urban-Environmental-Leeds/GEOG5415M-Programming-for-Spatial-Data-Science/blob/main/data/week_5/West_Yorkshire_bus_stops.geojson - https://github.com/MSc-Urban-Environmental-Leeds/GEOG5415M-Programming-for-Spatial-Data-Science/blob/main/data/week_5/leeds_travel_to_work_mode_distance.geojson

Remember, when we are reading in data from GitHub we need to amend the link from: '/blob/' to: '/raw/refs/heads/' in the code.

Run the code below to read the spatial data in

[ ]:
```
bus_stops =gpd.read_file('https://github.com/MSc-Urban-Environmental-Leeds/
 ↪GEOG5415M-Programming-for-Spatial-Data-Science/raw/refs/heads/main/data/
 ↪week_5/West_Yorkshire_bus_stops.geojson')
leeds=gpd.read_file('https://github.com/MSc-Urban-Environmental-Leeds/
 ↪GEOG5415M-Programming-for-Spatial-Data-Science/raw/refs/heads/main/data/
 ↪week_5/leeds_travel_to_work_mode_distance.geojson')
```

### 2.1.1 Data Exploration

Just as we have done before we are going to explore the data by having a look at the data frames and by visualising both the data sets.

[ ]:
```
# let's have a look at the bus_stops geopandas dataframe
bus_stops.head()
```

### 2.1.2 West Yorkshire Bus Stops

https://datamillnorth.org/dataset/vqxk4/west-yorkshire-bus-stops

The data includes: - 'STOPID': a unique ID for each bus stop - 'NAME': the name of each bus stop - 'TOWN': the town in which the bus stop is located - '0730_0930'...'2000_2100': average number of buses per hour during the AM peak (0730-0930), inter-peak (1200-1400), PM peak (1500-1830) and evening (2000-2100) time periods. - 'x' and 'y': coordinates (British National Grid eastings and northings, in metres) - 'z': elevation of the bus stop - 'PhotoF': link to a front photo of the bus stop - 'PhotoS': link to a side photo of the bus stop

- 'geometry': the type of geometry, in this case POINT Z, followed by the coordinates of the point. The Z means each point has three numbers: x, y and z (elevation).

[ ]:
```
# let's have a look at the Leeds geopandas dataframe
leeds.head()
```

### 2.1.3 Leeds

LSOA level polygon file for Leeds with 2021 census data about travel to work distance and transport mode, downloaded from geoportal: - Notice this time the 'geometry' column has more than two numbers and 'POLYGON()' rather than 'POINT()'

Find out how many bus stops there are in West Yorkshire, and how many LSOAs there are in Leeds Each row of a GeoDataFrame is one feature (one bus stop or one LSOA), so you can use .shape or len(), just as you did with ordinary pandas data frames in Weeks 2 and 3.

[6]:
```
# ANSWER
# how many bus stops are there? (rows, columns)
print(bus_stops.shape)

# how many LSOAs are there?
print(len(leeds))
```

```
(14255, 13)
488
```

## 2.2 Simple spatial data visualisation

### 2.2.1 Point data

[ ]:
```
# Plot the bus stop data

# define the plot size and number of subplots (1 i.e. 1 plot)
f, ax = plt.subplots(1, figsize=(16, 8))
# plot the bus stops, specifying the subplot axis
bus_stops.plot(ax=ax)
# show the plot
plt.show()
```

### 2.2.2 Polygon data

[ ]:
```
# Plot the leeds data

# define the plot size and number of subplots (1 i.e. 1 plot)
f, ax = plt.subplots(1, figsize=(16, 8))
# plot the leeds data, specifying the subplot axis
leeds.plot(ax=ax)
# show the plot
plt.show()
```

Note the geometry column this time is made up of POLYGON data, made up of many coordinate points. Let us have a closer look at the polygon geometry. We are going to look at the row with index 3.

[ ]:
```
# Let's look at the row with index 3 (the LSOA 'Leeds 009B')
leeds.loc[3,:]
```

[ ]:
```
# Now let us look at the geometry column
# Notice the shape is quite complex with lots of edges
leeds.loc[3,'geometry']
```

[ ]:
```
# let us look at the list of coordinates
print(leeds.loc[3,'geometry'])
# Each point is a corner of the LSOA
```

Now look at the geometry of the first bus stop (the row with index 0) in the same way, using bus_stops instead of leeds How is it different from the LSOA geometry? How many numbers are there in the coordinates of each bus stop?

[12]:
```
# ANSWER
# a bus stop is a single POINT with three coordinates (x, y and z), not a␣
 ↪POLYGON made of many points
print(bus_stops.loc[0, 'geometry'])
```

```
POINT Z (428983 428480 120)
```

### 2.2.3 geopandas.explore()

.explore() is a really useful function that generates an interactive Leaflet map from a GeoDataFrame. It is particularly useful for initial data exploration. The function creates: - a navigable map - with 'zoom in and out' functionality - and hover over function displaying the variables in the GeoDataFrame

The .explore() has many customisable parameters, see the package documentation

[ ]:
```
bus_stops.explore()
```

[ ]:
```
leeds.explore()
```

.explore() can also colour each feature by the values in one of the columns. To do this, add column='column name' inside the brackets. Make an interactive map of Leeds coloured by the number of people who cycle to work (the 'Bicycle' column) Hover over the LSOAs to check the values.

[15]:
```
# ANSWER
leeds.explore(column='Bicycle')
```

[15]:
```
[Interactive map produced by .explore(), not shown here. Run the cell to see
it.]
```

### 2.3 Coordinate Reference Systems (CRS)

Before we map multiple layers we need to check they have the same Coordinate Reference System (CRS). We can do this using the .crs attribute.

This displays useful information about the CRS, such as its units and the areas it can be used in.

Each CRS has a code in the EPSG registry. The three you will meet most often are: - EPSG:27700: British National Grid (units: metres). Use this for analysis in Great Britain. - EPSG:4326: WGS 84 latitude and longitude (units: degrees). Used by GPS and most web data. - EPSG:3857: Web Mercator (units: metres). Used by web maps and basemaps.

[ ]:
```
leeds.crs
```

Look at what happens when we change the projection

[ ]:
```
# NAD83 Canadian Spatial Reference System: Large and medium scale topographic␣
 ↪mapping and engineering survey.
leeds_new_proj= leeds.to_crs(epsg=2953)

# EPSG:3851 New Zealand Geodetic Datum 2000: Spatial referencing and conformal␣
 ↪mapping on the NZ continental shelf.
leeds_new_proj2= leeds.to_crs(epsg=3851)
```

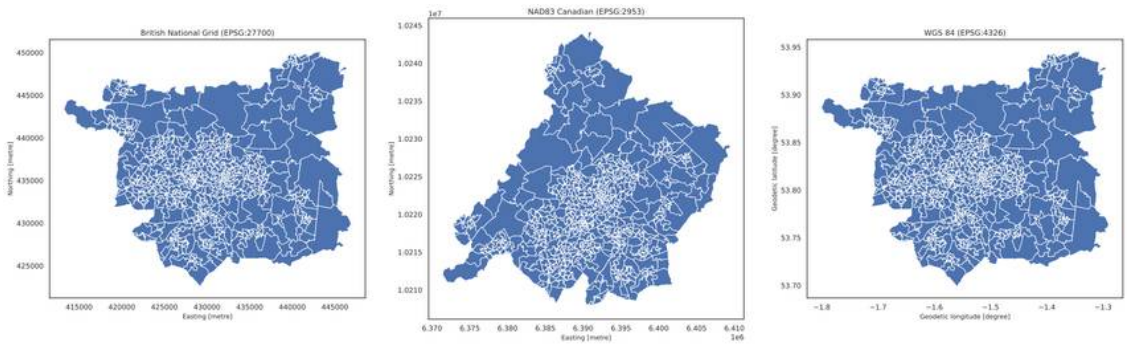
[ ]:
```
leeds_new_proj.crs
```

[ ]:
```
# plot the two different new projections alongside the original projection
f, ax = plt.subplots(1,3, figsize=(30, 20))
leeds.plot(ax=ax[0])
leeds_new_proj.plot(ax=ax[1])
leeds_new_proj2.plot(ax=ax[2])
plt.show()
# Note the different scales on the axis and orientation of the LSOAs
```

Convert the leeds data to WGS 84 latitude and longitude (EPSG:4326), save the result as leeds_wgs84, and plot it next to the original projection (`leeds`) and the Canadian projection (`leeds_new_proj`) Use the code above as a guide. Look at the numbers on the axes. What units do you think they are in now?

[20]:
```
# ANSWER
leeds_wgs84 = leeds.to_crs(epsg=4326)

f, ax = plt.subplots(1, 3, figsize=(30, 20))
leeds.plot(ax=ax[0])
leeds_new_proj.plot(ax=ax[1])
leeds_wgs84.plot(ax=ax[2])
ax[0].set_title('British National Grid (EPSG:27700)')
ax[1].set_title('NAD83 Canadian (EPSG:2953)')
ax[2].set_title('WGS 84 (EPSG:4326)')
plt.show()
# the WGS 84 axes are in degrees of longitude and latitude
```




### 2.4 Layering maps

Much like other mapping software it is possible to layer maps in Python. We will plot the bus stops on top of the leeds dataset.

We need to check the CRS of both datasets is the same, so we can accurately plot the layers on top of each other.

[ ]:
```
# Check the CRS are the same
print(leeds.crs, bus_stops.crs)
```

### 2.5 Plot the bus stops and LSOAs together

[ ]:
```
# plot one subplot (1 map), with dimensions 16 X 8
f, ax = plt.subplots(1, figsize=(16, 8))

# define the basemap plot it on the subplot axis
base = leeds.plot(ax=ax)

# plot the bus stops on the basemap axis, colour the bus stops red
bus_stops.plot(ax=base,color='red')
```

```
# show the map
plt.show()
```

As we can see we are able to plot the bus stop data (for the whole of West Yorkshire) onto the Leeds Map.

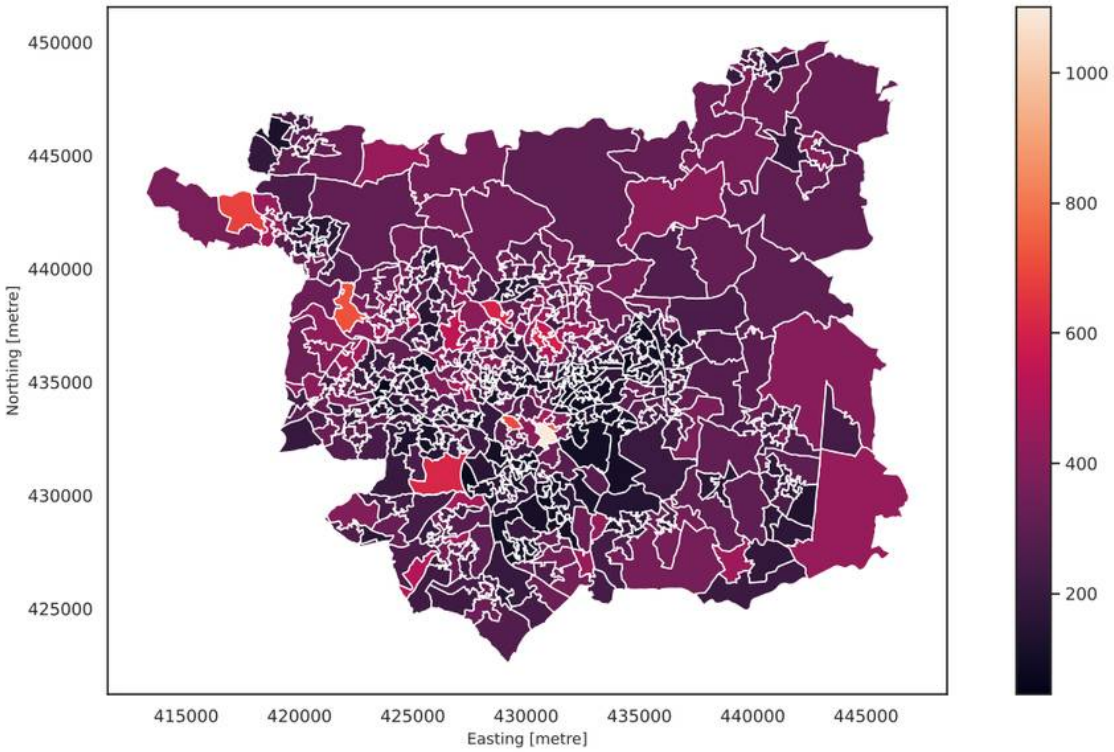
### 2.6 Choropleth Mapping

Choropleth maps are maps where the polygons are coloured different shades or colours based on a value. E.g. Number of people commuting by bus per LSOA

[ ]:
```
# Define plot size
f,ax = plt.subplots(1, figsize=(16,8))
# Plot the LSOAs, specifying to plot the 'Bus, minibus or coach' column
# Add legend (legend =True)
leeds.plot(ax=ax, column ='Bus, minibus or coach', legend=True)
# show the map
plt.show()
```

Code your own choropleth map for 'Works mainly from home'

[24]:
```
# ANSWER
f, ax = plt.subplots(1, figsize=(16, 8))
leeds.plot(ax=ax, column='Works mainly from home', legend=True)
plt.show()
```



### 2.6.1 Plotting Categorical variables

First let's generate a categorical variable. We might be interested in the most common way of travelling to work, other than by car, in each LSOA.

Have a look at the .idxmax() documentation: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.id

- what are we doing here?
- what does axis=1 do in the function?

Comment the code below to demonstrate you know what it is doing

[25]:
```
# ANSWER (commented version of the code)
# select only the columns for the non-car ways of travelling to work,
# then use .idxmax() to find, for each LSOA, the name of the column with the␣
 ↪largest value.
# axis=1 means we look across the columns of each row (rather than down each␣
 ↪column).
# The result (a column name for each LSOA) is stored in a new column called␣
 ↪'most_common_non_car_mode'
leeds['most_common_non_car_mode']=leeds[['Underground, metro, light rail, tram',
       'Train', 'Bus, minibus or coach', 'Taxi',
       'Motorcycle, scooter or moped', 'Bicycle', 'On foot']].idxmax(axis=1)
```

It is possible to create choropleth maps with categorical variables (e.g. most_common_non_car_mode). To do this we specify categorical=True, within the plot() function.

Note the legend of the map is now separate colours and not a continuous colour bar

Run the code below to plot a categorical choropleth

[ ]:
```
f,ax = plt.subplots(1, figsize=(16,8))
# plot the 'most_common_non_car_mode' column as categories
leeds.plot(ax=ax, column='most_common_non_car_mode', categorical=True,␣
 ↪legend=True)
plt.show()
```

Let's improve this visualisation a bit.

[ ]:
```
f,ax = plt.subplots(1, figsize=(16,8))
# plot the 'most_common_non_car_mode' column
# use the 'Set2' palette, which is designed for categorical data
leeds.plot(ax=ax, column ='most_common_non_car_mode', categorical=True,␣
 ↪cmap='Set2', legend=True)
# get the axis for the legend
leg = ax.get_legend()
# move the legend towards the bottom left corner
leg.set_bbox_to_anchor((0.3, 0.2))
```

```
# remove axis
ax.set_axis_off()
# set title
ax.set_title('Most common way of commuting, other than by car (LSOA)')
# show plot
plt.show();
```

leg.set_bbox_to_anchor((0.3, 0.2)) Try editing the coordinates in the set_bbox_to_anchor() function. See where the legend moves.

[ ]:
```
# list the wards in Leeds (we will use one of these in the next section)
leeds['WD24NM'].unique()
```

### 2.7 Sub-setting spatial data

We can also work with and plot just a subset of the spatial data. For example we might only be interested in the Ward of 'Headingley & Hyde Park' in Leeds. In which case we can use the .loc[] function to locate all rows (LSOAs) where the ward is 'Headingley & Hyde Park'. Just like we would normally do in a non-spatial pandas data frame.

Run the code below

[ ]:
```
# locate LSOAs (rows) in the Ward ('WD24NM') of 'Headingley & Hyde Park'
hyde_park = leeds.loc[leeds['WD24NM']== 'Headingley & Hyde Park',:]
# view the newly created hyde park dataframe
hyde_park
```

It is now possible to plot just Hyde Park and Headingley. We can also put maps side by side to compare them. Here we are going to compare the number of people commuting by bike and on foot in each LSOA.

Run the code below, make sure you understand what each line does (there is a lot going on)

[ ]:
```
# create a figure with two subplots (maps)
f,ax = plt.subplots(1,2, figsize=(12,6))

# plot the number commuting on foot in subplot 1
hyde_park.plot(ax=ax[0], column ='On foot', legend=True)

# plot the number commuting by bicycle in subplot 2
hyde_park.plot(ax=ax[1], column ='Bicycle', legend=True)

# give subplot 1 an informative title
ax[0].set_title('On foot')

# give subplot 2 an informative title
ax[1].set_title('Bicycle')

# make axis invisible for subplot 1
```

```
ax[0].set_axis_off()

# make axis invisible for subplot 2
ax[1].set_axis_off()

# show figure
plt.show()
```

[ ]:
```
## Let's improve the plots further

# create a figure with two subplots (maps)
f,ax = plt.subplots(1,2, figsize=(12,6))

# To compare the two maps fairly, they need to use the same colour scale.
# We do this by giving both plots the same minimum (vmin) and maximum (vmax) values.

# minimum = 0 (as these are counts)
vmin = 0
# calculate the maximum value across both columns
vmax = hyde_park[['On foot','Bicycle']].max().max()

# plot the number commuting on foot in subplot 1,
# shrink the legend and use the shared colour scale
hyde_park.plot(ax=ax[0], column ='On foot', legend=True, legend_kwds={'shrink':0.4}, vmin=vmin, vmax=vmax)

# plot the number commuting by bicycle in subplot 2,
# shrink the legend and use the shared colour scale
hyde_park.plot(ax=ax[1], column ='Bicycle', legend=True, legend_kwds={'shrink':0.4}, vmin=vmin, vmax=vmax)

# give subplot 1 an informative title
ax[0].set_title('Number of working population commuting on foot')

# give subplot 2 an informative title
ax[1].set_title('Number of working population commuting by bicycle')

# make axis invisible for subplot 1
ax[0].set_axis_off()

# make axis invisible for subplot 2
ax[1].set_axis_off()

# show figure
plt.show()
```

Choose a different ward from the list printed earlier. Create a subset of the leeds data for just that ward (call it my_ward), and make two maps side by side comparing the number of people who commute by bus ('Bus, minibus or coach') and by train ('Train') Copy the code above and edit it. Remember to change the titles as well.

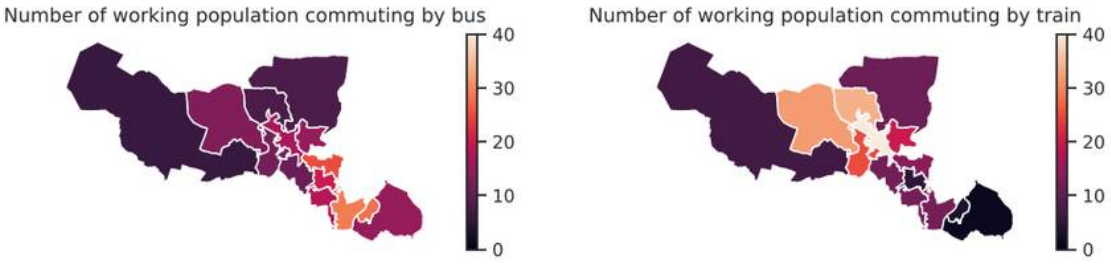
[32]:
```
# ANSWER (any ward will do, here we use Guiseley & Rawdon)
my_ward = leeds.loc[leeds['WD24NM'] == 'Guiseley & Rawdon', :]

f, ax = plt.subplots(1, 2, figsize=(12, 6))

# use the same colour scale for both maps
vmin = 0
vmax = my_ward[['Bus, minibus or coach', 'Train']].max().max()

my_ward.plot(ax=ax[0], column='Bus, minibus or coach', legend=True,␣
 ↪legend_kwds={'shrink': 0.4}, vmin=vmin, vmax=vmax)
my_ward.plot(ax=ax[1], column='Train', legend=True, legend_kwds={'shrink': 0.
 ↪4}, vmin=vmin, vmax=vmax)

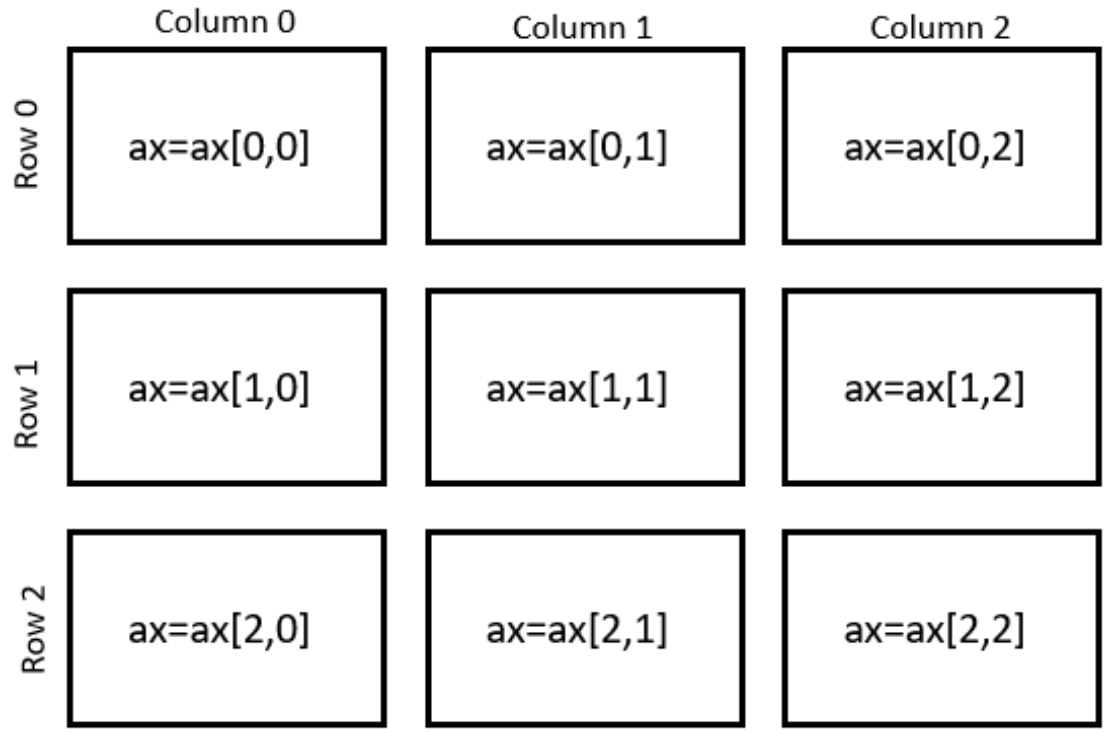
ax[0].set_title('Number of working population commuting by bus')
ax[1].set_title('Number of working population commuting by train')
ax[0].set_axis_off()
ax[1].set_axis_off()
plt.show()
```




## 2.8 Subplots and axes

A quick note on the subplot and ax indexing. It can be a little tricky getting your head around the indexing used when creating subplots. To create multiple subplots you use the code below, specifying firstly the number of rows then the number of columns. .subplots([number of rows], [number of columns])

When you are then specifying each individual plot, you need to specify the axes of the plot, using the code ax=ax[]. The indexing for the axes starts at 0. I.e. the first row is 'row 0' and the first column, 'column 0'. This is illustrated below.




For a set of subplots with just one row you only need to specify the column in the ax=ax[] function. I.e. ax=ax[column number]

I.e. f, ax = plt.subplots(1, 2) dataframe.plot(ax=ax[0], … dataframe.plot(ax=ax[1], …



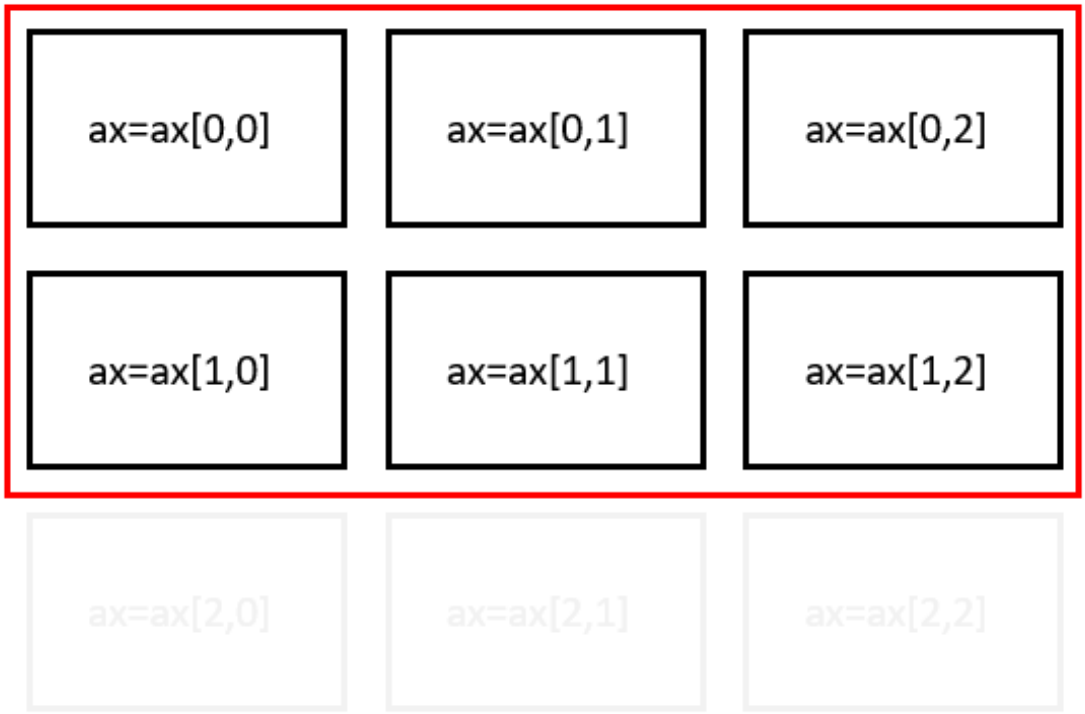

Similarly for a set of subplots with just one column you only need to specify the row in the

ax=ax[] function. f, ax = plt.subplots(2, 1) dataframe.plot(ax=ax[0], … dataframe.plot(ax=ax[1],




…

For a set of subplots with multiple rows and columns you need to specify both the row and the column (row first, then column): ax=ax[row, column] f, ax = plt.subplots(2, 3) dataframe.plot(ax=ax[0,0], … dataframe.plot(ax=ax[0,1], … dataframe.plot(ax=ax[0,2], … dataframe.plot(ax=ax[1,0], … dataframe.plot(ax=ax[1,1], … dataframe.plot(ax=ax[1,2], …




## 2.9 Independent coding task: Using USA data

### 2.9.1 Tasks:

- check the CRS of both geo data frames
- produce a layered map of cities and states
- produce a choropleth map of state population (the 'population' column)
- produce a map of the cities, coloured by elevation ('ELEV_IN_FT')
- colour the cities by state
- subset the data to plot a single state e.g. Washington

### 2.9.2 The data

[ ]:
```
# read the USA data directly from GitHub (these are example datasets from the␣
 ↪geoplot package)
usa_cities = gpd.read_file('https://raw.githubusercontent.com/ResidentMario/
 ↪geoplot-data/master/usa-cities.geojson')
contiguous_usa = gpd.read_file('https://raw.githubusercontent.com/ResidentMario/
 ↪geoplot-data/master/contiguous-usa.geojson')

# remove cities in states not in the contiguous USA (not connected directly to␣
 ↪the mainland), for ease of plotting
continental_usa_cities = usa_cities.loc[(usa_cities['STATE'] !="HI") &␣
 ↪(usa_cities['STATE'] !="AK" ) & (usa_cities['STATE'] !="PR")]
```

[ ]:
```
continental_usa_cities.head()
```

[ ]:
```
continental_usa_cities.plot()
```

[ ]:
```
contiguous_usa.head()
```

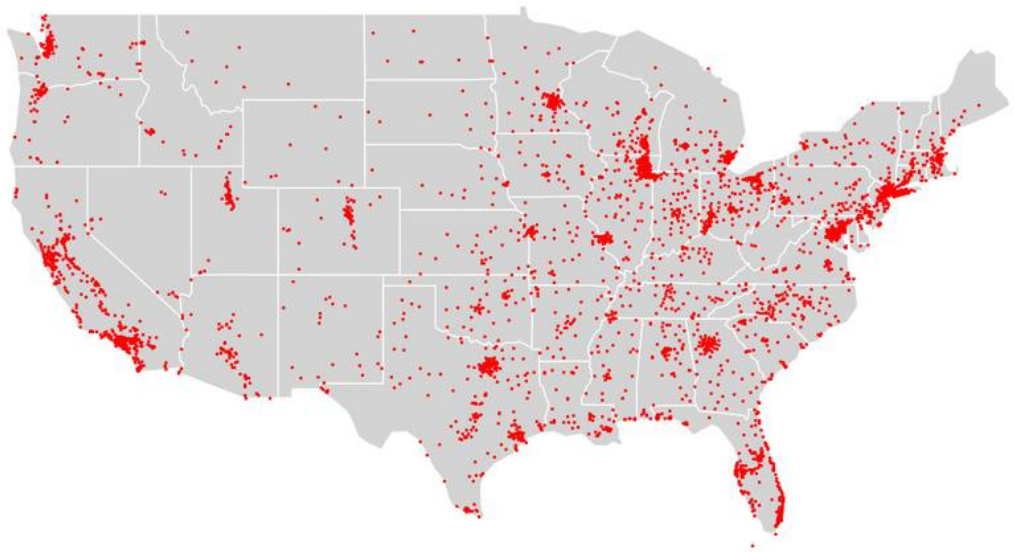
[ ]:
```
contiguous_usa.plot()
```

### 2.9.3 Answers

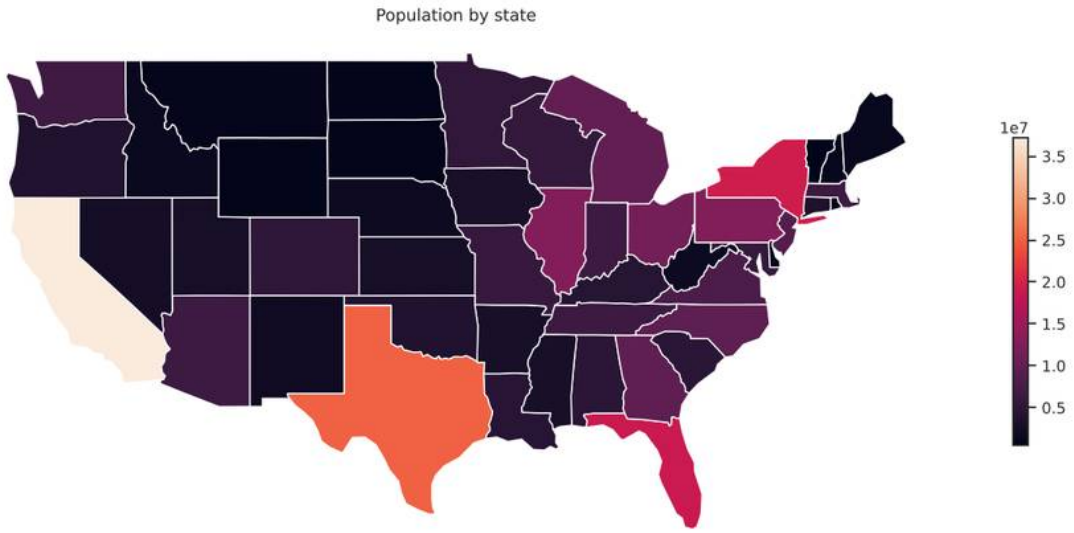
[38]:
```
# ANSWER: check the CRS of both geo data frames
print(continental_usa_cities.crs, contiguous_usa.crs)
```

```
EPSG:4326 EPSG:4326
```

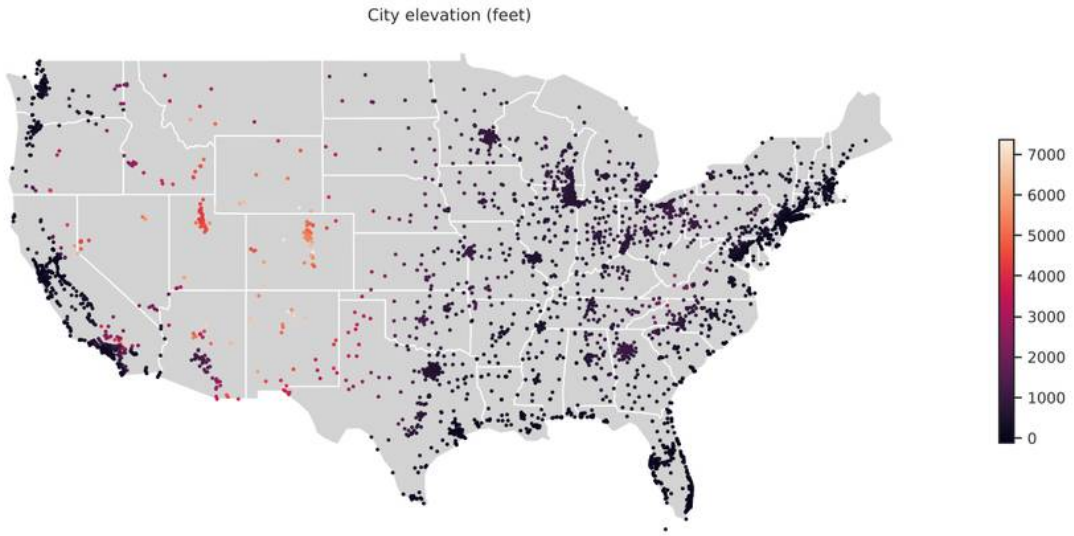
[39]:
```
# ANSWER: layered map of cities and states
f, ax = plt.subplots(1, figsize=(16, 8))
base = contiguous_usa.plot(ax=ax, color='lightgrey', edgecolor='white')
continental_usa_cities.plot(ax=base, color='red', markersize=2)
ax.set_axis_off()
plt.show()
```



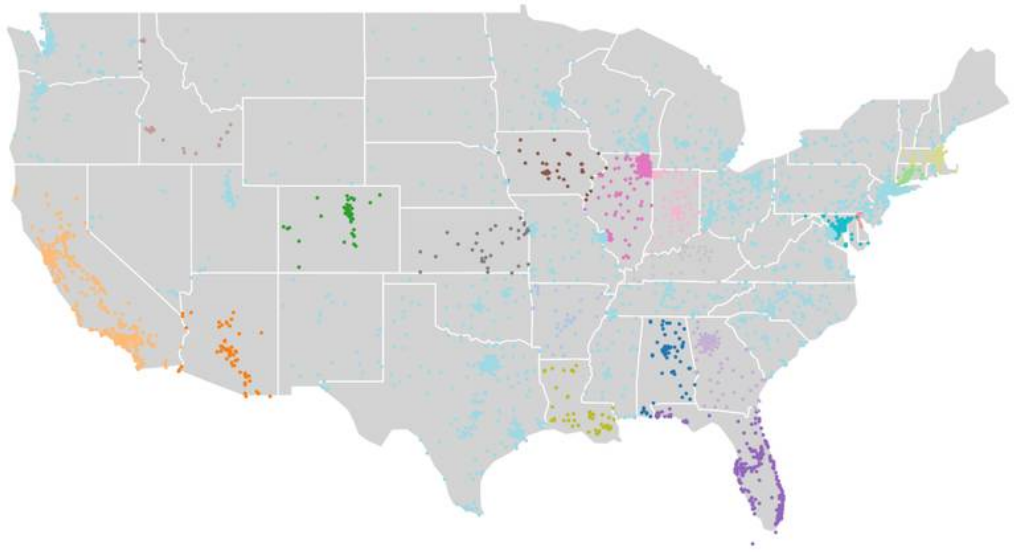
[40]:
```
# ANSWER: choropleth map of state population
f, ax = plt.subplots(1, figsize=(16, 8))
contiguous_usa.plot(ax=ax, column='population', legend=True,␣
 ↪legend_kwds={'shrink': 0.5})
ax.set_title('Population by state')
ax.set_axis_off()
plt.show()
```




[41]:
```
# ANSWER: cities coloured by elevation
f, ax = plt.subplots(1, figsize=(16, 8))
base = contiguous_usa.plot(ax=ax, color='lightgrey', edgecolor='white')
continental_usa_cities.plot(ax=base, column='ELEV_IN_FT', markersize=3,␣
 ↪legend=True, legend_kwds={'shrink': 0.5})
ax.set_title('City elevation (feet)')
ax.set_axis_off()
plt.show()
```

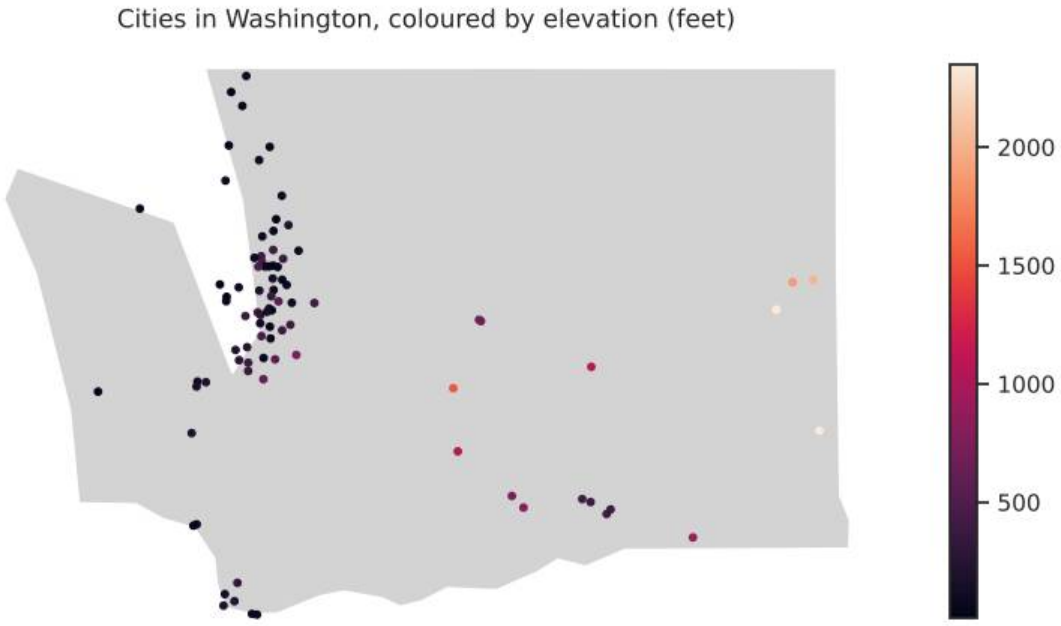


[42]:
```
# ANSWER: cities coloured by state
f, ax = plt.subplots(1, figsize=(16, 8))
base = contiguous_usa.plot(ax=ax, color='lightgrey', edgecolor='white')
continental_usa_cities.plot(ax=base, column='STATE', categorical=True,␣
 ↪cmap='tab20', markersize=3)
ax.set_axis_off()
plt.show()
```



[43]:
```
# ANSWER: subset the data to plot a single state (Washington)
# the state names are in the 'state' column of contiguous_usa, and the
# two-letter state codes are in the 'STATE' column of the cities data
washington = contiguous_usa.loc[contiguous_usa['state'] == 'Washington', :]
washington_cities = continental_usa_cities.loc[continental_usa_cities['STATE']␣
 ↪== 'WA', :]

f, ax = plt.subplots(1, figsize=(10, 8))
base = washington.plot(ax=ax, color='lightgrey')
washington_cities.plot(ax=base, column='ELEV_IN_FT', markersize=10,␣
 ↪legend=True, legend_kwds={'shrink': 0.6})
ax.set_title('Cities in Washington, coloured by elevation (feet)')
ax.set_axis_off()
plt.show()
```




# 3 Part B: Manipulating spatial data

## 3.1 Geometric Manipulations

[ ]:
```
#For the sake of this exercise we are going to change the projection of the data
bus_stops_p1 =bus_stops.to_crs('epsg:4326')
```

Let's compare CRS

[ ]:
```
bus_stops.crs
```

Notice that in the British National Grid (EPSG:27700) the axis information indicates the units used are metres:

Axis Info [cartesian]: - E[east]: Easting (metre) - N[north]: Northing (metre)

[ ]:
```
bus_stops_p1.crs
```

Notice that in the WGS 84 (World Geodetic System) geographic CRS (EPSG:4326) the axis information indicates the units used are degrees:

Axis Info [ellipsoidal]: - Lat[north]: Geodetic latitude (degree) - Lon[east]: Geodetic longitude (degree)

The CRS therefore matters when we come to do spatial analysis with our data (just like in every other GIS)

### 3.1.1 Buffers

A buffer in a geographic information system (GIS) is a zone around a map feature, measured in units of distance.

In Python we specify the buffer size: the radius of the buffer (in this case it is measured in metres as the CRS is EPSG:27700).

E.g.:

dataframe.buffer(distance =800) distance = 800 metres.

800 metres is often used to represent a 10-minute walk (400 metres is roughly a 5-minute walk).

[ ]:
```
# Plot 800 m buffers around bus stops

# plot one subplot (1 map), with dimensions 16 X 8
f, ax = plt.subplots(1, figsize=(16, 8))

bus_stops.buffer(distance=800).plot(ax=ax)

plt.show()
```

If the data were in EPSG:4326 the buffer distance would be in degrees. Run the code below and see what happens (geopandas also prints a warning that the CRS is geographic). We only plot the first five bus stops: one degree of latitude is about 111 km, so each 10-degree buffer is enormous!

[ ]:
```
bus_stops_p1.head(5).buffer(distance=10).explore()
```

[ ]:
```
# run the code without plotting- notice we generate an array of geometries
bus_stops.buffer(distance=800)
```

[ ]:
```
# we can check the data type- a geopandas array
bus_stops.buffer(distance=800).dtype
```

In order to keep information about the area being buffered in the newly created buffer geo-dataframe it is useful to copy the original data you want to conduct the geometric manipulation on, and name it something else e.g. dataframe_buffer.

Then replace the geometry column in the copied data with the calculated buffer geometry.

E.g. dataframe_buffer = dataframe.copy()

dataframe_buffer[‘geometry’] = dataframe.buffer(distance)

[ ]:
```
# copy the bus stops dataframe
bus_stops_buffer = bus_stops.copy()

# apply the function (replacing the geometry column with the buffer geometry)
bus_stops_buffer['geometry'] = bus_stops.buffer(distance=800)

bus_stops_buffer.head()
```

Note because we copied the bus_stops data frame above the bus_stops_buffer data frame still contains the name of the bus stop and other information e.g. average number of buses in the morning.

We can add the bus_stop_buffer layer to the map along with the bus_stop and leeds layers.

Run the code below, read the comments to understand what each line of code is doing

[ ]:
```
# plot one subplot (1 map), with dimensions 16 X 8
f, ax = plt.subplots(1, figsize=(16, 8))

# define the basemap plot it on the subplot axis
base = leeds.plot(ax=ax, color='grey')

# plot the bus stop buffers on the basemap axis, colour buffers blue
bus_stops_buffer.plot(ax=base,color='blue', alpha=0.2)

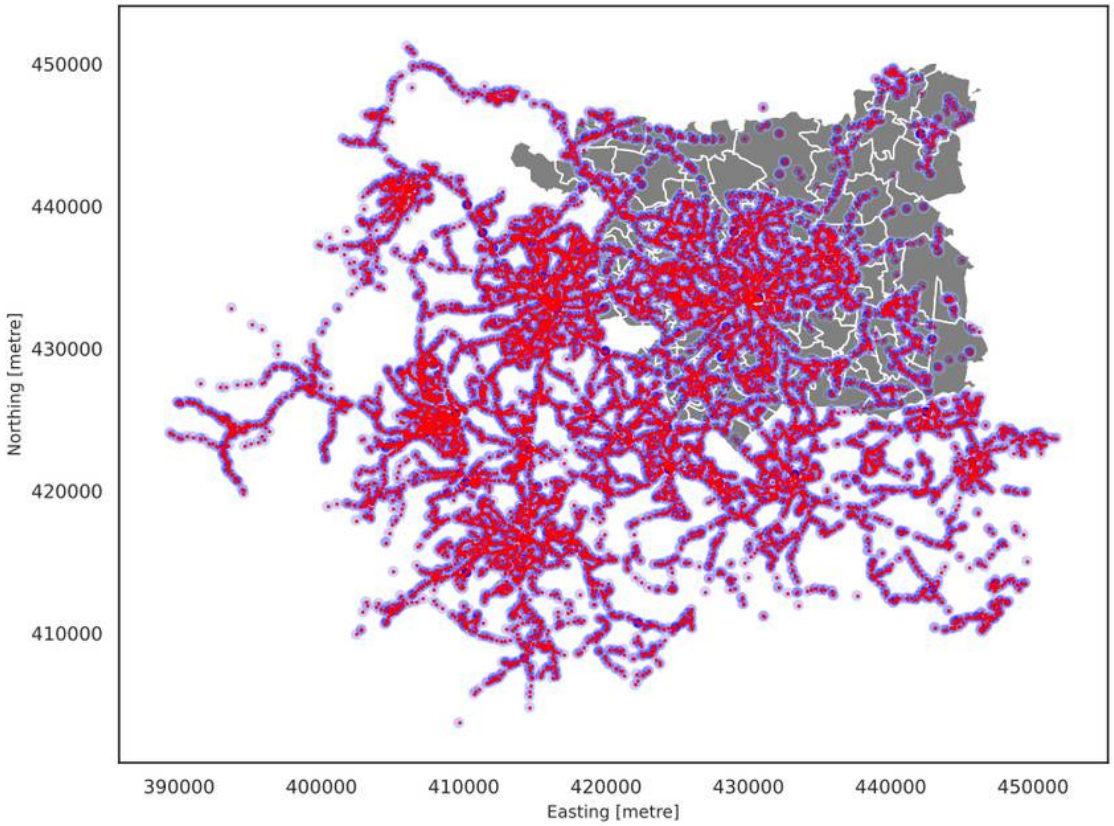
# plot the bus stops on the basemap axis, colour the bus stops red
bus_stops.plot(ax=base,color='red', markersize=1)

# show the map
plt.show()
```

Create 400 m buffers (roughly a 5-minute walk) around the bus stops, save them as bus_stops_buffer_400, and plot them on the map of Leeds in the same way as above How does the area within a 5-minute walk of a bus stop compare with the area within a 10-minute walk?

[53]:
```
# ANSWER
bus_stops_buffer_400 = bus_stops.copy()
bus_stops_buffer_400['geometry'] = bus_stops.buffer(distance=400)

f, ax = plt.subplots(1, figsize=(16, 8))
base = leeds.plot(ax=ax, color='grey')
bus_stops_buffer_400.plot(ax=base, color='blue', alpha=0.2)
bus_stops.plot(ax=base, color='red', markersize=1)
plt.show()
# much more of Leeds is now outside the buffers, particularly in rural areas
```




### 3.1.2 Centroids

Put simply, the centroid is the geometric centre of a polygon. For irregular shapes the centroid can fall outside the polygon. If you need a point that is guaranteed to be inside, use .representative_point() instead.

**Why calculate centroids?** Lots of geometric manipulations and analysis use centroids. For example you might use centroids as a proxy to measure the distance between two polygons.

Use the code dataframe.centroid to find the LSOA centroids (run the next 3 cells of code for the worked example below)

[ ]:
```
# copy the leeds dataframe
leeds_centroid = leeds.copy()

# calculate the centroid
# and replace the leeds geometry (polygon) with the centroid geometry (point)
leeds_centroid['geometry'] = leeds.centroid

# check - geometry should contain point data
leeds_centroid.head()
```

[ ]:
```
# plot to check it looks as expected
leeds_centroid.plot();
```

[ ]:
```
# plot one subplot (1 map), with dimensions 16 X 8
f, ax = plt.subplots(1, figsize=(16, 8))

# define the basemap plot it on the subplot axis
base = leeds.plot(ax=ax, color='grey')

# plot the lsoa centroids on the basemap axis, colour the centroids purple
leeds_centroid.plot(ax=base,color='purple')

# show the map
plt.show()
```

We can also change marker size and colour of plotted points to reflect the data they represent (as we did with the choropleth maps for polygon data).

Run the code below

[ ]:
```
# plot one subplot (1 map), with dimensions 16 X 8
f, ax = plt.subplots(1, figsize=(16, 8))

# define the basemap plot it on the subplot axis
base = leeds.plot(ax=ax, color='grey')

# plot the lsoa centroids on the basemap axis, colour and size the centroids␣
 ↪based on number of people commuting by bus
leeds_centroid.plot(ax=base, markersize='Bus, minibus or coach', column='Bus,␣
 ↪minibus or coach',  legend=True)

# make axis invisible
ax.set_axis_off()

# show the map
plt.show()
```

We can improve this visualisation further by scaling the marker size, adjusting the legend extent and adding a title

Run and read the code below, check you understand what is happening on each line

[ ]:
```
# plot one subplot (1 map), with dimensions 16 X 8
f, ax = plt.subplots(1, figsize=(16, 8))

# define the basemap plot it on the subplot axis
base = leeds.plot(ax=ax, color='grey')
```

```
# plot the lsoa centroids on the basemap axis, colour and size the centroids␣
 ↪based on number of people commuting by bus
# scale the marker size to fit better and make them semi-transparent to see␣
 ↪overlap in dense areas
# start colorbar legend at 0 as count data
leeds_centroid.plot(ax=base, markersize=leeds_centroid['Bus, minibus or coach']/
 ↪4, column='Bus, minibus or coach', alpha=0.8, legend=True, vmin=0)

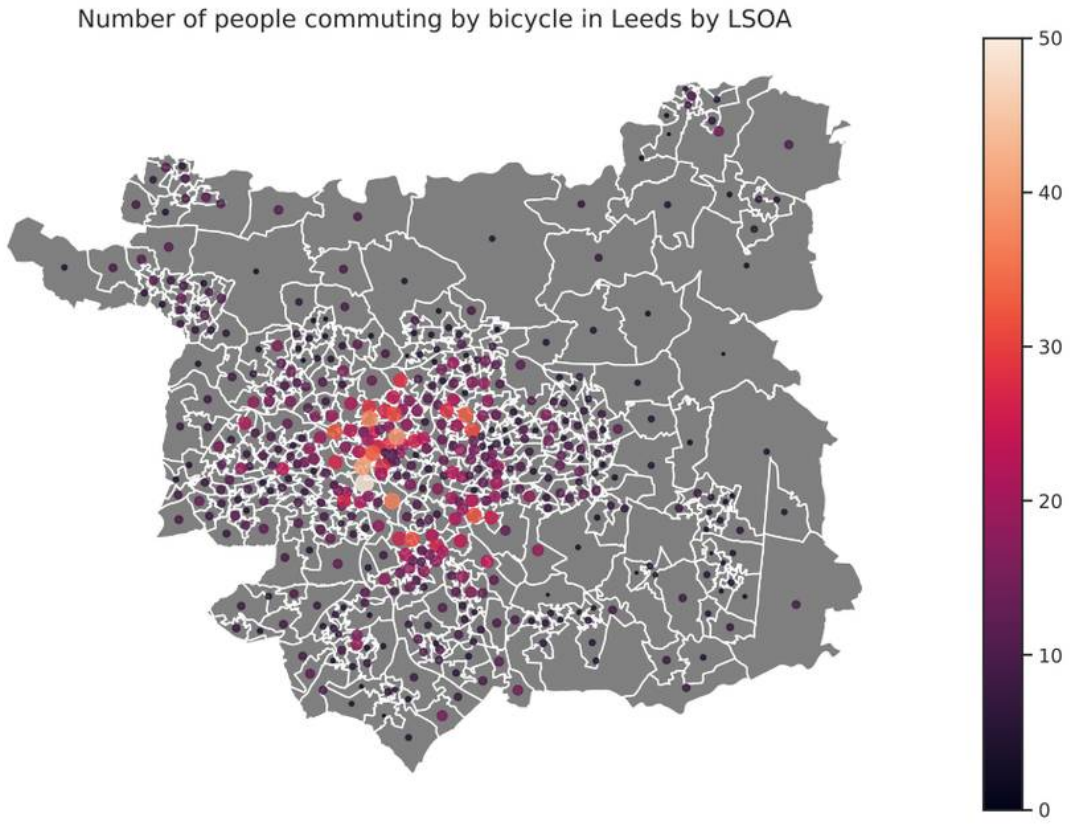
# make axis invisible
ax.set_axis_off()

# add title, increase fontsize
ax.set_title('Number of people commuting by bus in Leeds by LSOA', fontsize =14)

# show the map
plt.show()
```

Make a similar map showing the number of people who commute by bicycle in each LSOA Far fewer people cycle than take the bus, so dividing by 4 will make the circles very small. Try a different number (or multiply instead of dividing) until the circles are a sensible size.

[59]:
```
# ANSWER
f, ax = plt.subplots(1, figsize=(16, 8))
base = leeds.plot(ax=ax, color='grey')
# multiply rather than divide, as the numbers of cyclists are small
leeds_centroid.plot(ax=base, markersize=leeds_centroid['Bicycle']*2,␣
 ↪column='Bicycle', alpha=0.8, legend=True, vmin=0)
ax.set_axis_off()
ax.set_title('Number of people commuting by bicycle in Leeds by LSOA',␣
 ↪fontsize=14)
plt.show()
```




### 3.1.3 Convex hull polygons

A convex hull is the smallest polygon that you can draw around a collection of points/a polygon.

Run the code below to create convex hull polygons (using the code dataframe.convex_hull) around the LSOAs in Leeds

[ ]:
```
leeds_convex_hull = leeds.copy()

# replace the geometry column with the calculated convex hull polygon␣
 ↪geometries
leeds_convex_hull['geometry'] = leeds.convex_hull

# plot one subplot (1 map), with dimensions 16 X 8
f, ax = plt.subplots(1, figsize=(16, 8))

# define the basemap plot it on the subplot axis
base = leeds.plot(ax=ax, color='grey')

# plot the convex hull polygons
# .boundary allows us to just plot the polygon outline
```

```
leeds_convex_hull.boundary.plot(ax=base)

# make axis invisible
ax.set_axis_off()

# show the map
plt.show()
```

### 3.2 Part C: Spatially aggregating data

Often our data will not be at the same spatial scale, so we may need to aggregate areas of data together to get them to the same spatial scale. Or we may only be interested in larger spatial trends. Therefore, we need to convert our smaller area data to larger area data. In geopandas we can easily do this using the dissolve function. In this example we are going to aggregate LSOAs up to ward (WD24NM) level. Think of dissolve as removing all the internal LSOA borders within the ward to leave just the ward outline. The data for the ward also gets aggregated, below we have aggregated all the numeric data Run the code below

[ ]:
```
# specify which columns from the leeds dataframe we are going to aggregate:
# #this should include the column we are going to aggregate the data by (i.e.␣
 ↪WD24NM), the geometry column and any other columns we want to aggregate
# use the WD24NM column to inform the dissolve,
# in this case sum is a suitable aggregation to get the total within the wards,␣
 ↪we could also use 'mean' to get the average within the wards
leeds_wards = leeds[['WD24NM','Total_distance_to_work', 'Less than 10km',
       '10km to less than 30km', '30km and over', 'Works mainly from home',
       'Not in employment or works mainly offshore, in no fixed place or␣
 ↪outside the UK',
       'Work mainly at or from home', 'Underground, metro, light rail, tram',
       'Train', 'Bus, minibus or coach', 'Taxi',
       'Motorcycle, scooter or moped', 'Driving a car or van',
       'Passenger in a car or van', 'Bicycle', 'On foot',
       'Other method of travel to work',
       'Not in employment or aged 15 years and under','geometry']].
 ↪dissolve(by='WD24NM', aggfunc='sum').reset_index()

# view the new leeds_wards dataframe
leeds_wards
```

[ ]:
```
# plot the new leeds_wards df

leeds_wards.explore(column ='Bus, minibus or coach', vmin=0)
```

Think carefully about how you want to aggregate the columns. Counts (like the numbers here) can be summed, but percentages or rates cannot. We can also assign a different aggregation function to each column (as we did with .agg() when looking at the bike share data).

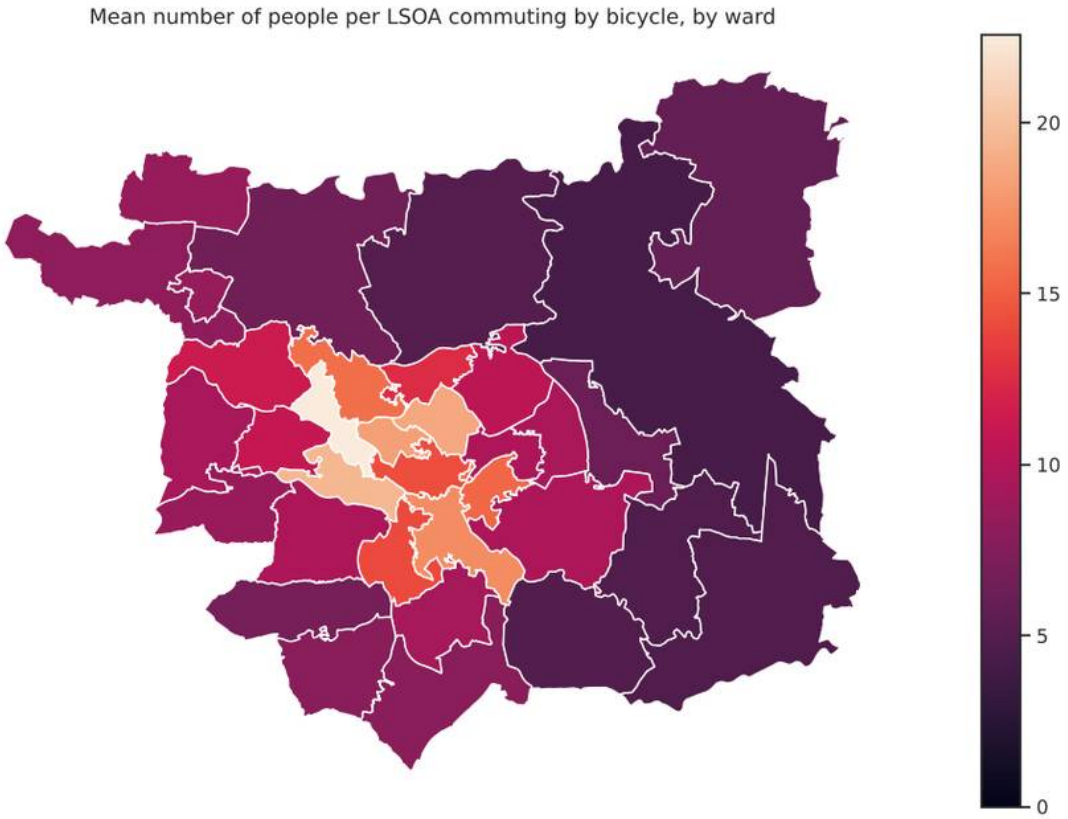
E.g. , aggfunc={'Bicycle':'sum', 'On foot':'mean'})

With a large dataset you might want to consider which columns you include in the aggregated spatial data frame.

**Replicate and edit the code above to aggregate the LSOA data to plot mean number of people commuting by bike per ward**

[63]:
```
# ANSWER
# aggregate the LSOAs to wards, taking the mean of the 'Bicycle' column
leeds_wards_bike = leeds[['WD24NM', 'Bicycle', 'geometry']].
 ↪dissolve(by='WD24NM', aggfunc='mean').reset_index()

f, ax = plt.subplots(1, figsize=(16, 8))
leeds_wards_bike.plot(ax=ax, column='Bicycle', legend=True, vmin=0)
ax.set_title('Mean number of people per LSOA commuting by bicycle, by ward')
ax.set_axis_off()
plt.show()
```




## 3.3 Part D: Spatial Data Joins

## 3.4 Joining a non-spatial dataset to a spatial dataset

We are going to read in some data about the resident population of LSOAs across England and Wales and join it to the Leeds data frame

Run the code below to read in a csv of LSOA level resident population data

```
[ ]: # read in residential population data from 2021 census
lsoa_resident_pop =pd.read_csv('https://github.com/
 ↪MSc-Urban-Environmental-Leeds/GEOG5415M-Programming-for-Spatial-Data-Science/
 ↪raw/refs/heads/main/data/week_5/lsoa_resident_pop.csv')
# use loc to subset data to only contain data for Leeds
leeds_resident_pop =lsoa_resident_pop.loc[lsoa_resident_pop['geography'].str.
 ↪contains('Leeds'),:]
# have a quick look at the dataframe
leeds_resident_pop.head()
```

```
[ ]: print('Check both datasets have the same number of rows:', leeds_resident_pop.
 ↪shape[0], leeds.shape[0])
```

```
[ ]: leeds.head()
```

The 'LSOA21CD' column in the Leeds data frame matches that of the 'geography code' column in the leeds_resident_pop data frame. As they are formatted in exactly the same way and each row is a unique LSOA, the geography code/ LSOA21CD is a unique identifier common to both data frames. Therefore, we will use this column to join our datasets. The code we will use is pd.merge(). We need to specify a few parameters within the function:

- Firstly we specify which data frames we want to join (in the order we want to join them). pd.merge(leeds, leeds_resident_pop…
- Secondly we need to specify the column in each data frame, leeds is on the left so we specify left_on='LSOA21CD': as we are using the 'LSOA21CD' column from the leeds dataframe. And the leeds_resident_pop data frame is on the right, so we specify right_on='geography code' as we are using the 'geography code' column for our join. pd.merge(leeds, leeds_resident_pop, left_on='LSOA21CD', right_on='geography code' …
- Finally, we need to specify how the tables are joined. This is based on SQL join types (see the pandas guide to merging). In this case we are using a left join (we keep all the data in the left data frame and just add the columns from the right dataframe on the end). pd.merge(leeds, leeds_resident_pop, left_on='LSOA21CD', right_on='geography code', how='left')

Note: put the spatial data frame (leeds) on the left. The result takes its type from the left-hand data frame, so if the non-spatial data frame were on the left the result would be an ordinary DataFrame rather than a GeoDataFrame, and .plot() would not draw a map.

```
[ ]: leeds_pop = pd.merge(leeds, leeds_resident_pop, left_on='LSOA21CD',␣
 ↪right_on='geography code', how='left')
leeds_pop.head()
```

```
[ ]: # check if any of the rows in the newly joined columns are NA, if so this might␣
 ↪suggest that the join has not worked as expected
leeds_pop['Residence type: Total; measures: Value'].isna().any()
```

Now we will visualise our newly joined data by plotting the total resident population in each LSOA

[ ]:
```
f,ax = plt.subplots(1, figsize=(16,8))
leeds_pop.plot(column='Residence type: Total; measures:␣
 ↪Value',legend=True,ax=ax)
# remove axis
ax.set_axis_off()
plt.show();
```

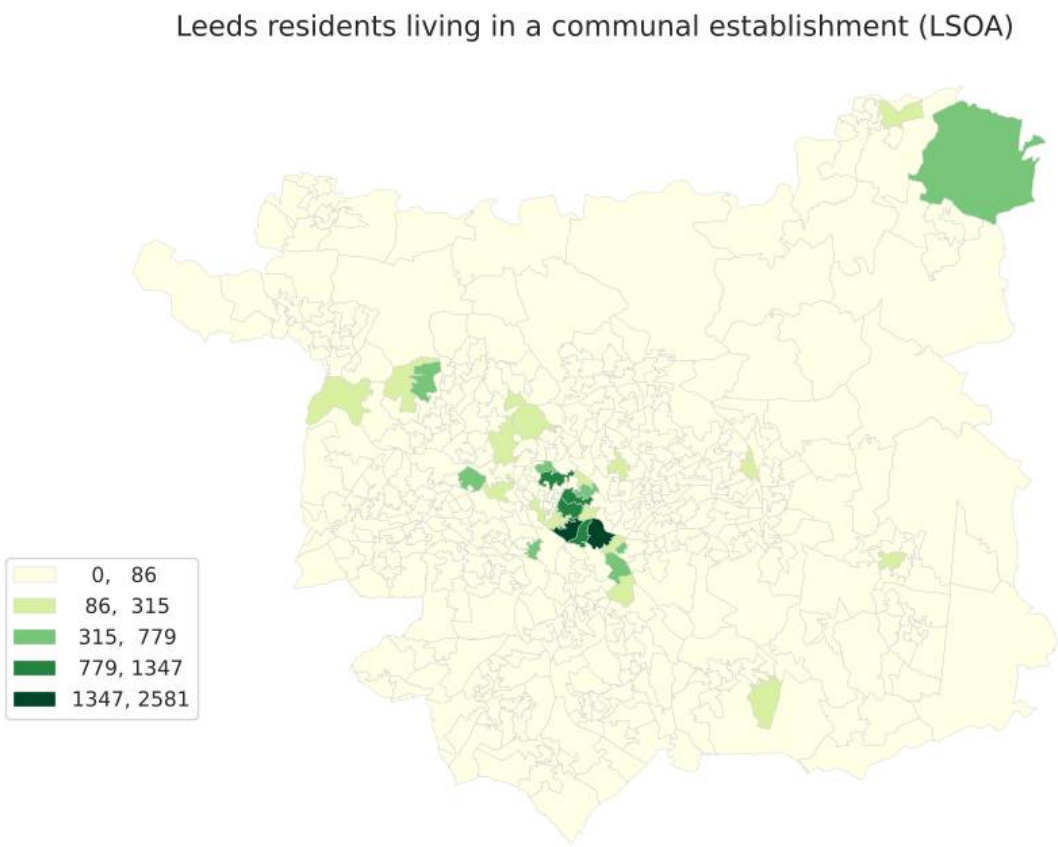
With a continuous colour scale it can be difficult to tell areas apart, especially when a few areas have much larger values than the rest. To make a more informative map we can group the values into classes using scheme=' ', and set the number of classes with k. Common schemes include ['equal_interval', 'quantiles', 'natural_breaks', 'fisher_jenks']. With 'quantiles', each class contains the same number of LSOAs. (These schemes come from the mapclassify package that we installed at the start.)

[ ]:
```
f,ax = plt.subplots(1, figsize=(16,8))
# classify the data into 5 quantiles, show the class limits as whole numbers in␣
 ↪the legend,
# and draw thin grey boundaries so the palest areas are still visible
leeds_pop.plot(column='Residence type: Total; measures: Value',legend=True,␣
 ↪cmap='YlGn',scheme='quantiles', k=5, legend_kwds={'fmt': '{:.0f}'},␣
 ↪edgecolor='lightgrey', linewidth=0.3, ax=ax)
# remove axis
ax.set_axis_off()
# position legend
ax.get_legend().set_bbox_to_anchor((.12, .4))
# add title
ax.set_title('Leeds total resident population (LSOA)', fontsize=15)
# add a North arrow (look at the documentation here: https://github.com/
 ↪pmdscully/geo_northarrow)
add_north_arrow(ax=ax, scale=.75, xlim_pos=0.1, ylim_pos=.85, color='#000',␣
 ↪text_scaler=2, text_yT=-1.25)
plt.show();
```

Write your own code to map 'Residence type: Lives in a communal establishment; measures: Value'

[71]:
```
# ANSWER
f, ax = plt.subplots(1, figsize=(16, 8))
# most LSOAs have nobody living in a communal establishment, so quantiles␣
 ↪cannot make 5 classes.
# natural breaks (Fisher-Jenks) works better here
leeds_pop.plot(column='Residence type: Lives in a communal establishment;␣
 ↪measures: Value', legend=True, cmap='YlGn',
                scheme='fisher_jenks', k=5, legend_kwds={'fmt': '{:.0f}'},␣
 ↪edgecolor='lightgrey', linewidth=0.3, ax=ax)
ax.set_axis_off()
ax.get_legend().set_bbox_to_anchor((.12, .4))
```

```
ax.set_title('Leeds residents living in a communal establishment (LSOA)',␣
 ↪fontsize=15)
plt.show()
```




### 3.5 Spatially Joining Data

It is also possible to join data based on their spatial relationship to each other using gpd.sjoin(). For example we might want to know the LSOA each bus stop is in.

Like with pd.merge() there are parameters we need to specify when using gpd.sjoin()

- Again firstly we specify which data frames we want to join (in the order we want to join them). gpd.sjoin(bus_stops, leeds, …
- Secondly we need to specify how the tables are joined: 'inner', 'left' or 'right' (again based on SQL join types). In this case we are using an inner join, so only the bus stops that fall within an LSOA in Leeds are kept. gpd.sjoin(bus_stops, leeds, how="inner",…
- Finally, we need to specify the spatial relationship (the 'predicate') that decides whether two features are joined. These are the relationships illustrated in the lecture slides. The most common are:

- intersects: the features share any part of space (they touch or overlap). This is the default.
- within: the feature in the left data frame lies entirely inside the feature in the right data frame.
- contains: the feature in the left data frame completely contains the feature in the right data frame.

Others include 'touches', 'crosses' and 'overlaps'. See the geopandas documentation for the full list.

Run the code below to spatially join the data

[ ]:
```
bus_stops_leeds = gpd.sjoin(bus_stops, leeds, how="inner",␣
 ↪predicate='intersects')
# look at the data: notice that we now have the LSOA level info for each bus␣
 ↪stop
bus_stops_leeds.head()
```

Note when mapped we now just have bus stops within Leeds, not the whole of West Yorkshire

[ ]:
```
# let us plot the bus stops now coloured by the number of people commuting by␣
 ↪bus in that LSOA
f,ax = plt.subplots(1, figsize=(16,8))

# map the leeds LSOAs (basemap)
base = leeds.plot(ax=ax, color='grey')
# plot the bus stops and colour them based on n of people commuting by bus in␣
 ↪the LSOA in which the bus stop is
bus_stops_leeds.plot(ax=base,  column ='Bus, minibus or coach',␣
 ↪cmap='viridis',markersize=2, vmin=0,legend=True)
# remove axis
ax.set_axis_off()
plt.show()
```

Use gpd.sjoin() to find the bus stops in the Headingley & Hyde Park ward (the hyde_park data frame we created earlier). Save the result as hyde_park_bus_stops How many bus stops are there in the ward? Then plot the bus stops on top of a map of the ward.

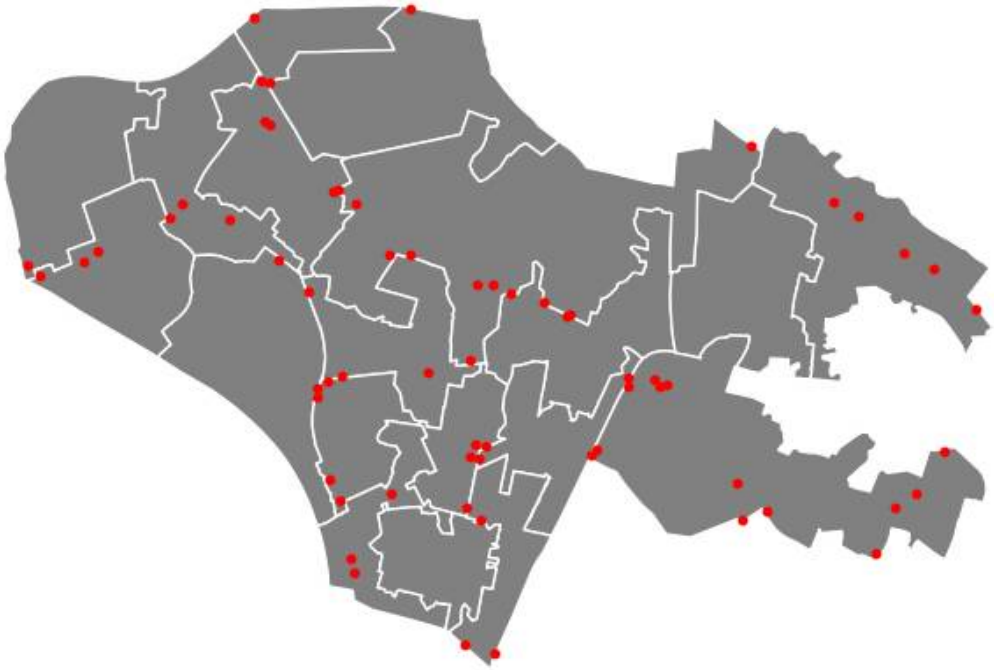
[74]:
```
# ANSWER
# find the bus stops in Headingley & Hyde Park
hyde_park_bus_stops = gpd.sjoin(bus_stops, hyde_park, how='inner',␣
 ↪predicate='intersects')

# how many are there?
print(len(hyde_park_bus_stops))

# plot them on top of a map of the ward
f, ax = plt.subplots(1, figsize=(8, 8))
base = hyde_park.plot(ax=ax, color='grey')
```

```
hyde_park_bus_stops.plot(ax=base, color='red', markersize=10)
ax.set_axis_off()
plt.show()
```

65



**Overlay - creating spatial layers from the intersections, unions, and differences between maps** I have included this for reference with a worked example, to show you how we can look at where spatial data overlaps. A list of the overlay operations can be found here: https://geopandas.org/en/stable/docs/user_guide/set_operations.html Have a read and explore the data.

[ ]:
```
# get the intersection between the bus stop buffers (a 10-minute walk) and the␣
↪LSOAs,
# to see which parts of each LSOA are within a 10-minute walk of a bus stop
lsoas_in_buff = gpd.overlay(bus_stops_buffer, leeds, how='intersection')
# look at the data
lsoas_in_buff
```

[ ]:
```
# plot the results
f,ax = plt.subplots(1, figsize=(16,8))

# lsoa base map
```

```
base = leeds.plot(ax=ax,color='grey')
# plot the intersection of bus stop buffer and lsoas
lsoas_in_buff.plot(ax=base, alpha=0.8)
# remove axis
ax.set_axis_off()
plt.show();
```

## 3.6 Part E: Kernel density maps and basemaps

### 3.6.1 Kernel density map

Much like the kernel density plots we produced in Week 4, this produces a kernel density estimate of spatial point data: a smooth surface showing where the points are concentrated. The bw_adjust parameter of sns.kdeplot() controls how smooth the surface is.

[ ]:
```
## Seaborn example

f,ax = plt.subplots(1, figsize=(16,8))

# map the leeds (basemap)
base = leeds.plot(ax=ax, color='grey')

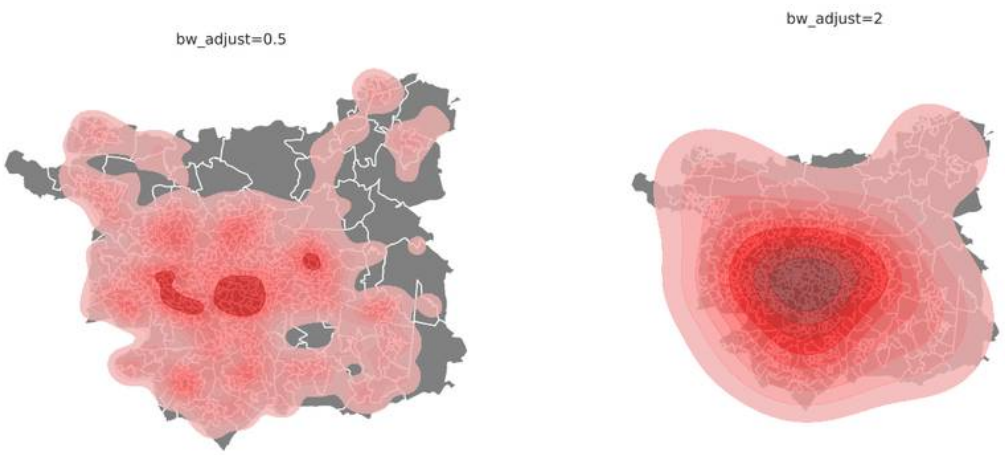
# map the kernel density estimate of bus stops
sns.kdeplot(bus_stops_leeds, x='x', y='y', color='Red', fill=True,ax=base,␣
 ↪alpha=0.8)

# remove axis
ax.set_axis_off()

plt.show()
```

Copy the code above and add the bw_adjust parameter to sns.kdeplot(). Try bw_adjust=0.5 and then bw_adjust=2 What happens to the map? (If you leave it out, bw_adjust is 1.)

[78]:
```
# ANSWER
# smaller values give a less smooth surface that shows more local detail,
# larger values give a smoother surface
f, ax = plt.subplots(1, 2, figsize=(16, 8))
for i, bw in enumerate([0.5, 2]):
    base = leeds.plot(ax=ax[i], color='grey')
    sns.kdeplot(bus_stops_leeds, x='x', y='y', color='Red', fill=True, ax=base,␣
 ↪alpha=0.8, bw_adjust=bw)
    ax[i].set_title(f'bw_adjust={bw}')
    ax[i].set_axis_off()
plt.show()
```




### 3.6.2 Adding base maps

Base maps provide the reader with context for a map. When zoomed in further, the basemap may show road networks or points of interest.

We are going to use contextily to add the background map to the geographic data using the add_basemap() function.

**Aligning the CRS** But first we need to convert the CRS of the data we want to plot to Web Mercator (EPSG:3857), so that the base maps and the geographic data we are plotting align.

Unless the data file I want to plot is particularly big I tend to save the data with the Web Mercator CRS as a new geodataframe to avoid confusion.

[ ]:
```
leeds.crs
```

[ ]:
```
# convert CRS to epsg=3857 Web Mercator
leeds_WM = leeds.to_crs(epsg=3857)

fig,ax = plt.subplots(1, figsize=(10,10))

# plot the Leeds data as usual
leeds_WM.plot(column='Works mainly from home',ax=ax,alpha=0.5)

# add ctx basemap to ax (by default contextily uses OpenStreetMap Humanitarian␣
↪tiles)
ctx.add_basemap(ax, crs=leeds_WM.crs)

ax.set_axis_off()
```

**Different base maps.** By default ctx.add_basemap() uses OpenStreetMap Humanitarian tiles.

To see the other available base maps we can get the provider keys:

List of contextily providers:

[ ]:
```
ctx.providers.keys()
```

Each will have their own options, accessed by the code: ctx.providers.provider_from_above.keys()

E.g. ctx.providers.Esri.keys()

Some providers (for example CartoDB and Stadia) now require a free API key. You can check with, e.g., ctx.providers.CartoDB.Voyager.requires_token()

[ ]:
```
ctx.providers.Esri.keys()
```

We can then use the provider and provider options to choose our base map e.g. Esri.WorldGrayCanvas

In the first example below we plot the data in its original CRS (British National Grid) and pass crs=leeds.crs to add_basemap(). This tells contextily to reproject the map tiles to match our data (the tiles can look slightly blurred as a result). In the second example we use the Web Mercator version of the data, so we do not need to give a crs.

[ ]:
```
f,ax = plt.subplots(1, figsize=(16,16))

# plot the Leeds data as usual
leeds.plot(alpha=0.5,column='Works mainly from home',ax=ax)

# add ctx basemap to ax, specifying the basemap provider and options
ctx.add_basemap(ax,source=ctx.providers.Esri.WorldGrayCanvas, zoom=12,␣
 ↪crs=leeds.crs)

ax.set_axis_off()
```

[ ]:
```
f,ax = plt.subplots(1, figsize=(16,16))

# plot the Leeds data as usual
leeds_WM.plot(alpha=0.5,column='Works mainly from home',ax=ax)

# add ctx basemap to ax, specifying the basemap provider and options
ctx.add_basemap(ax,source=ctx.providers.Esri.WorldImagery, zoom=12)

ax.set_axis_off()
```

Play around with the providers and options to get different base maps.

[ ]:
```
# ANSWER (one example: OpenStreetMap tiles)
f, ax = plt.subplots(1, figsize=(16, 16))
leeds_WM.plot(alpha=0.5, column='Works mainly from home', ax=ax)
ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik, zoom=12)
ax.set_axis_off()
```